

HABITQUEST

Developer Study & Learning Guide

A Zero-Dependency Vanilla PWA & Electron Architecture

Author: Antigravity Pair Program

Published: May 2026

0<ß" HABITQUEST: DEVELOPER STUDY & LEARNING GUIDE

Welcome to your comprehensive project study guide! This document is designed as a structured curriculum to help you master how **HabitQuest** was built from scratch. It explains every architectural layer, design decision, and code module in a clear, educational way.

0=Þàþ The Tech Stack: Zero-Dependency & Pure Vanilla

Many modern web apps are built with complex frameworks (React, Vue, Angular) and heavy libraries (Chart.js, Tailwind, Webpack). For **HabitQuest**, we deliberately chose a **100% Zero-Dependency Vanilla Architecture**:

1. ****HTML5 (Structure):**** Semantic structure featuring tab panels, native forms, dynamic SVGs, and dialog modals.
 2. ****Vanilla CSS3 (Aesthetic Design):**** Flexbox, CSS Grid, animations (`@keyframes``), viewport media queries, and custom styling variables (`--cat-health``, `--bg-secondary``) to build the warm typewriter parchment notebook theme.
 3. ****Vanilla JS / ES6 Modules (System Logic):**** Clean, modular scripts divided by concerns using native `import`` and `export`` statements without requiring complex transpilation or bundling.
 4. ****LocalStorage (Database):**** Client-side, persistent browser database that operates instantaneously and operates 100% offline.
 5. ****Service Workers (Mobile PWA Shell):**** Intercepts network fetch requests, caches static assets, and permits full screen mobile app installations.
 6. ****Electron (Desktop Shell):**** A Chromium wrapper that compiles your local web files into a standalone native Windows `.exe`` desktop application.
-

0=ÜÐ Systems Architecture & File Organization

HabitQuest operates as a **Single Page Application (SPA)**. Instead of reloading pages, it dynamically renders updates directly to the DOM

based on a modular structure:

- ****index.html****: The static canvas.
- ****js/app.js****: App entry point and service worker bootstrapper.
- ****js/storage.js****: Data model, backup manager, and state synchronizer.
- ****js/gamification.js****: Calculations for leveling, streaks, and award criteria.
- ****js/charts.js****: Renders high-fidelity interactive SVG graphics using plain math.
- ****js/ui.js****: Listens for UI clicks, updates DOM nodes, and triggers re-renders.

0=ÝÃþ Data Management & Storage Engine (storage.js)

All habits and logs are serialized into text strings and stored securely in your browser's physical memory using the **LocalStorage API**.

```
{
  "id": "hq-9f8d7c6-l3a9z8y",
  "name": "Pushups",
  "category": "health",
  "frequency": { "type": "daily", "days": [] },
  "habitId": "hq-9f8d7c6-l3a9z8y",
  "createdAt": "2026-05-26",
  "completed": true,
  "note": "Felt strong! Did 30 reps."
}
```

Data Schemas

1. ****Habit Object****
2. ****Log Object****

0>Ýà Core Concept: Date Normalization

Time zones are a major bug source in calendar trackers. If you log a habit at 11:30 PM, a time zone shift could push the date to "tomorrow." HabitQuest resolves this by standardizing all log records to local <2>YY-MM-DD + stayings via a utility function:

```
export function formatDate(date) {
  const d = new Date(date);
  let month = d.getMonth() + 1;
  let day = d.getDate();
  const year = d.getFullYear();
  if (month.length < 2) month = `0${month}`;
  if (day.length < 2) day = `0${day}`;
  return [year, month, day].join('-');
}
```

0=ÜÈ Math & SVG Visualization Engine

(charts.js)

Rather than importing a large 200KB graphing framework, HabitQuest draws graphics dynamically using **Vector Graphics (SVG)**.

Today's circular completion Progress:

We calculate the circumference of a circle ($2 * \pi$

```
* r) and adjust the `stroke-dashoffset` in real  
time.  
const radius = 60;  
const circumference = 2 * Math.PI * radius; // Approx 377px  
const strokeDashOffset = circumference * (100 - progress) / 100;  
const strokeDashArray = [377];  
<circle cx="70" cy="70" r="60" stroke-dasharray="377" stroke-das
```

```
</svg>
```

By binding this inline in an SVG tag:

The browser immediately draws a perfect half-circle completion line without any layout performance drop!

Ø<BÆ Gamification Mechanics (gamification.js)

Gamifying real life keeps motivation high. HabitQuest manages character statistics using algebraic leveling rules:

1. ****Target XP per level:**** Target XP increases by 100 XP per level:

$$\text{TargetXP} = \text{Level} * 100$$

2. ****Reward Multipliers:**** Base completions earn `10 XP`. If a user maintains a continuous daily streak, they gain a cumulative bonus to reward consistency:

$$\text{XPReward} = 10 + \text{Math.min}(25, \text{Streak})$$

Ø=Üñ Responsive Layout & Mobile Protection (main.css)

Mobile responsiveness is about flexibility

and bounding sizes:

1. ****Flex Sizing & Bounding (min-width: 0):**** Flexbox items default to their minimum content size (`min-width: auto`). To let long text text wrap cleanly instead of stretching cards off the screen, we enforce

```
.habit-details { min-width: 0; }
.habit-name { word-break: break-word; white-space: normal; }
```

2. ****Fluid Aspect-Ratio Checkboxes:****

On narrow screens (Samsung S23 portrait), fixed-size checkbox grids overflow. Enforcing an aspect-ratio grid fits perfectly:

```
checkboxes {
  width: 100%;
  max-width: 380px;
  aspect-ratio: 1 / 1;
}
```

3. ****Overlay Modal Scrolling:**** To prevent modal forms from clipping on short viewports, the parent overlay uses

```
.modal-overlay { align-items: flex-start; overflow: auto; margin: 10px auto; max-height: 100%; }
```

& Progressive Web App Caching Logic (service-worker.js)

PWAs install local assets to bypass network requests for offline access. This requires custom caching and update lifecycles:

1. ****Cache Versioning (CACHE_NAME):**** Incrementing the version string forces the browser to discard old cache assets during the `activate` event.
2. ****Ignore Search Matching (ignoreSearch):**** Setting `{ ignoreSearch: true }` ignores style version query strings (like `?v=6.0`) during runtime matches, keeping files cached for offline runs.
3. ****Hot-Reload Listener (app.js):**** Reloads the browser frame once a new cache is active, pushing

```
        if (newWorker.state === 'installed') {
            window.location.reload();
        }
    });
    design.fixes immediately;
```



